

Maintenance d'un entrepôt de données

VenteRapide, entrepôt PostgreSQL et dbt (schéma en étoile de Kimball)

Auteur	Anice GUIREN
Certification	Data Engineer (RNCP 37638), compétences C16 et C17
Contexte	Data engineer junior chez ShopFlow, pour le client VenteRapide
Date	Septembre 2026

Ce rapport retrace la mise en conditions opérationnelles de l'entrepôt de VenteRapide, de la compréhension de l'existant jusqu'à sa documentation, en passant par ses évolutions et son exploitation au quotidien.

Sommaire

1. Introduction et contexte
2. Comprendre l'entrepôt existant
3. Intégrer une nouvelle source de données
4. Gérer les variations de dimensions (SCD)
5. Organiser la maintenance
6. Superviser l'activité, journaux et tableau de bord
7. Sauvegarder et restaurer
8. Gérer les accès et respecter le RGPD
9. Documenter et préparer la montée en charge
10. Les modèles logique et physique
11. Passage en production, ce qui changerait
12. Conclusion et retour d'expérience

Annexes

1. Introduction et contexte

VenteRapide est une boutique en ligne généraliste présente sur le marché français depuis 2021. L’entreprise dispose déjà d’un entrepôt de données opérationnel qui rassemble au même endroit ses commandes, ses produits, ses clients et les visites de son site. Chaque jour, l’équipe analytique s’appuie sur cet entrepôt pour produire ses reportings. En tant que data engineer junior chez ShopFlow, l’entreprise de services qui accompagne VenteRapide, ma mission n’est donc pas de construire cet entrepôt mais de le maintenir et de le faire évoluer en conditions opérationnelles.

Le maintien en conditions opérationnelles signifie qu’un système déjà en production doit continuer à fonctionner de façon fiable dans la durée. Concrètement, quelqu’un doit surveiller son activité, sauvegarder ses données, gérer les droits d’accès, intégrer les nouvelles sources et faire évoluer sa structure sans jamais le casser. C’est ce que recouvrent les deux compétences évaluées, la C16 pour l’exploitation et la maintenance au sens large, la C17 pour la gestion des variations dans les dimensions.

Le rapport suit le déroulé réel du projet. Il commence par la compréhension de l’entrepôt existant, car on ne modifie pas un système sans l’avoir compris, présente ensuite les deux évolutions apportées, décrit l’exploitation au quotidien en sécurité, puis termine par la documentation et les modèles de données à jour. Les captures qui prouvent chaque résultat sont regroupées en annexe, et le texte y renvoie par des liens cliquables.

2. Comprendre l’entrepôt existant

La pile technique

L’entrepôt repose sur un ensemble d’outils, rappelés ici car ils reviennent tout au long du rapport.

Élément	Technologie	Rôle
Base de données	PostgreSQL 16	stocke et sert les données
Transformation (ETL)	dbt 1.11	construit les modèles en SQL
Modélisation	schéma en étoile (Kimball)	organise les faits et les dimensions
Planification et exploitation	cron, bash, msmtpt	automatise les tâches et les alertes
Versioning	Git	historise le code
Environnement	Linux (Ubuntu 24.04)	héberge l’ensemble

Un entrepôt organisé en couches successives

Le point le plus important à saisir est que dbt ne mélange pas tout. Il fait circuler la donnée à travers des couches qui ont chacune un rôle défini, et c’est cette organisation qui rend l’entrepôt lisible et fiable.

La première couche est constituée des seeds. Ce sont les fichiers CSV bruts, chargés tels quels dans des tables préfixées par `raw_`. À ce stade rien n’est transformé, ce qui garantit une traçabilité complète de la donnée d’origine. La deuxième couche est le staging. Pour chaque source, une vue préfixée par `stg_` nettoie et type la donnée, par exemple en convertissant une date stockée en texte vers une vraie date. Cette couche joue le rôle de sas de qualité. La troisième couche contient le cœur analytique, avec les dimensions qui décrivent le contexte, comme `dim_client` ou `dim_produit`, et les faits qui contiennent les événements chiffrés, comme `fact_commandes`. Cette organisation de faits entourés de dimensions est précisément le schéma en étoile, détaillé dans la partie 10. Enfin, un snapshot nommé `scd_client` conserve l’historique des changements du client, dont le fonctionnement est expliqué dans la partie 4.

Remettre l'environnement en route

Avant toute intervention, j'ai vérifié que l'entrepôt fonctionnait réellement en installant PostgreSQL et dbt puis en lançant la construction complète du projet avec la commande `dbt build`, qui enchaîne le chargement des seeds, la construction des modèles, le snapshot et l'exécution des tests. La base cohabitant avec un autre service sur le port habituel, l'instance du projet a simplement été placée sur un autre port. La construction s'est ensuite déroulée sans erreur et l'ensemble des tests est passé au vert, ce qui me donnait un point de départ sain avant de faire évoluer quoi que ce soit. La preuve de cette mise en route figure en [annexe A](#).

3. Intégrer une nouvelle source de données

Le besoin métier

VenteRapide souhaite analyser les retours de produits afin de comprendre quels articles sont le plus souvent renvoyés et pour quel motif. Cette information existait dans un fichier `raw_retours.csv` mais restait en dehors de l'entrepôt, donc inexploitable par les analystes. Mon travail a consisté à l'y intégrer proprement.

Faire passer la donnée par les mêmes couches

Le principe de la maintenance est de ne jamais bricoler une solution parallèle. J'ai donc fait entrer les retours par le même chemin que le reste des données, avec les trois commandes suivantes.

```
dbt seed --select raw_retours           # crée la table brute raw_retours
dbt run  --select stg_retours fact_retours # nettoie puis construit le fait
dbt test --select stg_retours fact_retours # exécute les tests associés
```

La première commande charge le fichier comme un nouveau seed. La deuxième construit la vue de staging `stg_retours`, qui type notamment la date du retour et le montant remboursé, puis la table de faits `fact_retours`. Le choix de modélisation de cette table mérite d'être expliqué. Sa granularité est d'une ligne par retour. Le fait ne recopie pas les informations du client ou du produit, il se contente de porter les clés vers les dimensions déjà existantes, à savoir `dim_client`, `dim_produit` et `dim_date`. On appelle cela réutiliser des dimensions conformes, et c'est la bonne pratique, car cela garantit qu'un client signifie la même chose dans toutes les analyses. La référence à la commande est conservée dans le fait, le motif du retour reste un attribut, et le montant remboursé constitue la mesure que l'on pourra agréger.

Garantir l'intégrité par les tests

Chaque nouveau modèle a été accompagné de tests. Les plus importants sont les tests de relation, qui vérifient que chaque clé étrangère du fait pointe bien vers une valeur existante dans sa dimension. Autrement dit, ils garantissent qu'aucun retour ne fait référence à un client ou à un produit fantôme. La construction complète de l'entrepôt s'est terminée avec cent huit opérations réussies et aucune erreur, et l'analyse des retours est désormais possible, comme le montre l'[annexe B](#).

Cette étape a réservé une difficulté parlante. Un des motifs possibles est la valeur `Changement d'avis`, qui contient une apostrophe. Or le test qui vérifie la liste des valeurs autorisées entoure chaque valeur de guillemets simples en SQL. L'apostrophe interne cassait donc la requête et faisait échouer le test. La correction consiste à doubler l'apostrophe, ce qui est la manière normale d'échapper ce caractère en SQL. Au delà de l'anecdote, cet incident illustre l'intérêt des tests, puisqu'ils ont intercepté une erreur avant qu'elle n'atteigne la production.

4. Gérer les variations de dimensions, les SCD (C17)

Le problème posé par le temps

Cette partie répond à la compétence C17. Le problème est simple à énoncer. Les attributs d'une dimension changent au fil du temps. Un client déménage, un produit change de prix. La question centrale, formalisée par Ralph Kimball, est de savoir ce que l'on fait de l'ancienne valeur quand la nouvelle arrive. Selon la réponse, on parle de dimension à variation lente de type 1, 2 ou 3.

Type	Comportement au changement	Historique conservé	Exemple dans le projet
Type 1	la nouvelle valeur écrase l'ancienne	aucun	le prix dans <code>dim_produit</code>
Type 2	une nouvelle ligne est créée, l'ancienne est fermée par des dates de validité	complet	la ville et le segment dans <code>scd_client</code>
Type 3	l'ancienne valeur est gardée dans une colonne dédiée	partiel, un seul niveau	non utilisé ici

Le bon choix dépend du besoin. On retient le type 2 quand l'historique a une vraie valeur pour l'analyse, et le type 1 quand seule la valeur actuelle compte.

Le type 2 sur le client

L'entrepôt historise déjà le client grâce au snapshot `scd_client`, qui surveille les colonnes de ville, de code postal et de segment. Quand l'une d'elles change, dbt ferme l'ancienne version en lui donnant une date de fin, puis ouvre une nouvelle version. Pour le démontrer, j'ai simulé un déménagement du client numéro 1 de Bordeaux vers Paris, puis relancé le snapshot. Le client possède alors deux lignes, la version Bordeaux archivée avec une date de fin renseignée, et la version Paris courante dont la date de fin est vide. Cette historisation est visible en [annexe C](#).

L'intérêt réel du type 2 apparaît quand on veut retrouver l'état du client au moment d'un événement passé. J'ai écrit pour cela une requête qui joint les commandes et le snapshot en s'appuyant sur les bornes de validité. Elle rattache chaque commande à la version du client dont l'intervalle de validité contient la date de la commande. On constate ainsi que les commandes de 2023 de ce client ressortent avec Bordeaux, la ville où il habitait alors, et non avec Paris, sa ville actuelle. Sans le type 2, on réécrirait l'histoire à chaque changement et les rapports passés deviendraient faux. Le résultat de cette requête figure en [annexe C](#).

Le type 1 sur le produit

Le cas du prix est différent, car on ne cherche pas à conserver l'ancien tarif, seule la valeur courante nous intéresse. La table `dim_produit` étant reconstruite entièrement à chaque exécution de dbt, elle réalise nativement un comportement de type 1. Je l'ai vérifié en modifiant le prix d'un produit dans la source. Après reconstruction, la table porte le nouveau prix, la marge est recalculée, et il n'existe toujours qu'une seule ligne pour ce produit, sans aucune trace de l'ancien tarif, comme le montre [l'annexe C](#).

Un enseignement important de cette partie est qu'aucune modification du code de transformation n'a été nécessaire pour gérer ces deux variations. Le snapshot assure de lui-même le type 2 et la table assure de lui-même le type 1. Les variations s'intègrent donc sans dénaturer le schéma en étoile initial, ce qui répond directement à l'exigence de la compétence C17.

5. Organiser la maintenance

Une fois l'entrepôt enrichi, il faut l'exploiter dans la durée de façon organisée. Cette partie pose le cadre des suivantes.

La méthodologie retenue

Je me suis appuyé sur trois approches complémentaires. La première est le référentiel ITIL, qui structure la gestion d'un service informatique en cinq étapes, la stratégie des services, leur conception, leur transition, leur exploitation et l'amélioration continue. J'en retiens surtout la distinction entre la gestion des incidents, qui vise à rétablir le service au plus vite, et la gestion des problèmes, qui traite la cause profonde pour éviter la récurrence. La deuxième approche est le ticketing, qui trace chaque demande ou incident sous la forme d'un ticket doté d'une priorité, d'un responsable et d'un statut. La troisième est la méthode Kanban, issue des méthodes agiles, qui rend

le flux de travail visible sur un tableau allant de la colonne à faire jusqu’à la colonne terminé, en limitant le nombre de tâches menées en parallèle.

La priorisation et la répartition

Les tâches sont classées selon une matrice qui croise leur impact et leur urgence, ce qui donne quatre niveaux de priorité. Le délai indiqué est celui de la prise en charge, c’est-à-dire le moment où quelqu’un commence à traiter le ticket.

Priorité	Situation	Exemple concret	Délai de prise en charge
P1 critique	service indisponible ou données corrompues	entrepôt injoignable, sauvegarde échouée	moins d’une heure
P2 majeur	fonction dégradée mais contournable	construction dbt nocturne en échec	moins de deux heures
P3 mineur	gêne sans blocage	requête lente, documentation à mettre à jour	moins d’un jour ouvré
P4 mineur non urgent	amélioration ou dette technique	renommage d’un modèle	moins d’une semaine

Chaque type de tâche est confié à un rôle précis. Le data engineer prend en charge les modèles et les tests dbt. L’administrateur de la base s’occupe des sauvegardes, des accès et du stockage. Le référent chargé des données personnelles suit le registre et les purges. L’analyste remonte les anomalies qu’il constate. Un incident critique déclenche l’intervention conjointe du data engineer et de l’administrateur.

Les indicateurs de service et leur intérêt

Prendre un ticket en charge rapidement ne suffit pas, encore faut-il mesurer la qualité réelle du service rendu. C’est le rôle des indicateurs de service, chacun associé à une cible que l’on s’engage à tenir, ce que l’on appelle un accord de niveau de service ou SLA. Ces indicateurs mesurent la résolution complète et la santé générale de l’entrepôt, et non plus seulement le démarrage du traitement. Par exemple, le délai moyen de résolution d’un incident critique est plus long que son délai de prise en charge, puisqu’il couvre le diagnostic et la correction.

Indicateur	Cible visée	Valeur d’exemple	État
Disponibilité mensuelle de l’entrepôt	au moins 99,5 %	99,8 %	conforme
Délai moyen de résolution d’un incident critique	moins de 4 heures	1 heure 20	conforme
Fraîcheur des données	moins de 24 heures	6 heures	conforme
Taux de réussite des constructions dbt	au moins 99 %	100 %	conforme
Taux de réussite des sauvegardes sur sept jours	100 %	100 %	conforme
Tests dbt au vert	100 %	87 sur 87	conforme

Ce tableau de bord est alimenté automatiquement à partir des journaux de la base, du fichier de résultats produit par dbt et des mesures de la machine, à l’aide de l’outil de visualisation Grafana. Sa

mise en place concrète fait l'objet de la partie suivante.

6. Superviser l'activité, journaux et tableau de bord

Superviser l'entrepôt revient à répondre à trois questions. Que s'est-il passé, comment être prévenu quand quelque chose se casse, et comment garder son état sous les yeux en permanence. J'ai répondu sur deux niveaux complémentaires et hiérarchisés. Le premier est réactif et détaillé, il repose sur les journaux et sur une alerte par courriel, et sert à comprendre un incident une fois qu'il survient. Le second est proactif et visuel, il repose sur des mesures affichées dans un tableau de bord, et sert à surveiller les tendances pour réagir avant la panne. Les deux ne se remplacent pas, ils se complètent, et le second se nourrit même du premier.

Le niveau réactif, journaux et alerte

PostgreSQL classe chaque événement selon un niveau de gravité, du plus bavard au plus grave. Pour conserver au minimum les alertes et les erreurs, j'ai ajouté un petit fichier de configuration, inclus automatiquement par la base, contenant les réglages suivants.

```
log_min_messages = warning      # journalise les alertes (WARNING) et au dessus
log_min_error_statement = error  # ajoute la requête responsable de chaque erreur
log_connections = on            # trace les connexions à la base
```

Ces réglages s'appliquent par un simple rechargement à chaud, sans redémarrer la base ni couper le service. Chaque ligne du journal porte alors son niveau de gravité, ce qui permet de retrouver les alertes et les erreurs d'un simple filtre.

```
sudo systemctl reload postgresql@16-main
grep -E "WARNING:|ERROR:" /var/log/postgresql/postgresql-16-main.log
```

Pour vérifier le dispositif, j'ai provoqué volontairement une alerte puis une erreur, et je les ai retrouvées dans le journal, correctement catégorisées et accompagnées de la requête fautive, comme le montre l'annexe E.

Il fallait aussi être prévenu quand la reconstruction de l'entrepôt échoue. Un script lance la construction dbt et examine son code de sortie, le nombre que renvoie tout programme pour dire s'il a réussi. Un code de zéro laisse le script silencieux. Un code différent déclenche un courriel contenant les dernières lignes du journal, pour comprendre tout de suite l'origine du problème. L'envoi passe par un utilitaire de messagerie dont les identifiants restent dans un fichier protégé, jamais écrits dans le script. Testé sur un échec réel de la construction, le dispositif a bien déclenché la réception du courriel, visible en annexe E.

Le niveau proactif, métriques et tableau de bord

Les journaux et l'alerte répondent après coup, une fois l'incident survenu. Ils ne donnent pas l'état de l'entrepôt d'un seul coup d'œil, ni l'évolution de ses indicateurs dans le temps, ni le moyen de réagir avant la rupture. C'est le rôle d'un second niveau, que j'ai monté avec Prometheus et Grafana, les deux outils annoncés dans la partie 5 pour alimenter le tableau de bord de service.

Le principe tient en trois rôles. Des exporteurs exposent des mesures sur une page web, l'un pour la machine et l'autre pour PostgreSQL. Prometheus vient lire ces pages à intervalle régulier et conserve l'historique des mesures. Grafana interroge Prometheus et affiche le tout. L'ensemble est décrit dans un fichier docker compose et démarre d'une seule commande.

Restait à y faire entrer nos propres indicateurs, le résultat du build dbt et la date des sauvegardes, qui ne sont connus que de nos scripts. Plutôt que de refaire ce travail, je l'ai raccordé au niveau réactif. Les scripts de construction et de sauvegarde écrivent désormais, en plus de leur journal, un petit fichier de mesures que l'exporteur de la machine republie vers Prometheus. Le même script qui envoie l'alerte en cas d'échec alimente ainsi le tableau de bord, ce qui montre bien la hiérarchie entre les deux niveaux, le réactif produit le détail et l'alerte, le proactif en tire une vue d'ensemble.

Le tableau de bord réunit alors sur une seule vue les indicateurs de service de la partie 5, l'état de la base et de la machine, le succès du dernier build dbt, la fraîcheur des données et l'âge de la dernière

sauvegarde. Chaque mesure change de couleur selon des seuils, ce qui fait ressortir une dérive avant qu'elle ne devienne une panne. Le tableau de bord est présenté en [annexe G](#).

7. Sauvegarder et restaurer

Un entrepôt en production doit pouvoir être restauré en cas d'incident, qu'il s'agisse d'une corruption, d'une fausse manipulation ou d'une panne de disque. La règle d'or du métier est qu'une sauvegarde qui n'a jamais été testée ne vaut rien.

La stratégie de sauvegarde

J'ai mis en place une sauvegarde logique à l'aide de l'outil `pg_dump`, qui produit un fichier contenant à la fois la structure des tables et l'intégralité des données. Perdre la base ne perd donc rien tant que l'on possède ce fichier.

```
pg_dump -Fc -f dw_full_${date +%F}.dump datawarehouse_e6
```

J'ai défini deux niveaux de sauvegarde. La sauvegarde complète porte sur toute la base, s'exécute chaque nuit et est conservée quatorze jours. La sauvegarde partielle porte uniquement sur les tables de faits, qui sont les plus volatiles, s'exécute toutes les six heures et est conservée trois jours. Ce découpage limite la perte de données possible entre deux sauvegardes complètes. Ces deux tâches sont planifiées avec cron, l'ordonnanceur de Linux, qui les exécute automatiquement même lorsque aucune session n'est ouverte.

```
0 2 * * * backup.sh full      # sauvegarde complète chaque nuit à 2 heures
0 */6 * * * backup.sh partial  # sauvegarde partielle toutes les 6 heures
```

Les identifiants nécessaires sont rangés dans un fichier de mots de passe dédié, ce qui évite d'écrire le moindre secret dans les scripts ou dans la planification.

Le test de restauration

Surtout, j'ai réellement testé la restauration, car c'est la seule preuve valable. Le test restaure la dernière sauvegarde complète dans une base jetable, compte les lignes récupérées pour vérifier que le contenu est bien présent, puis supprime cette base de test. Le résultat, avec le décompte exact des lignes restaurées, figure en [annexe D](#).

8. Gérer les accès et respecter le RGPD

Cette partie comporte une action technique et plusieurs livrables de conformité, car la gestion des accès touche directement à la protection des données personnelles.

Un accès en lecture seule selon le moindre privilège

Le principe du moindre privilège consiste à n'accorder que les droits strictement nécessaires. L'équipe analytique a besoin de lire l'entrepôt, mais jamais de le modifier. J'ai donc créé un rôle nommé `reporting` disposant uniquement du droit de lecture et d'aucun droit d'administration.

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO reporting;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO reporting;
```

La seconde ligne est importante, car elle accorde aussi la lecture sur les tables qui seront créées plus tard, sans quoi chaque nouveau modèle serait invisible pour ce rôle. J'ai ensuite vérifié le rôle de deux manières complémentaires, ce qui est la preuve attendue. En positif, une requête de lecture fonctionne. En négatif, toute tentative d'écriture est refusée. Ces deux vérifications sont visibles en [annexe F](#).

La stratégie d'accès et le registre des traitements

La stratégie retenue repose sur des accès nominatifs limités au besoin réel, sur la traçabilité des connexions assurée par la journalisation, et sur la minimisation des données exposées. Lorsqu'un

usage n'a pas besoin des colonnes identifiantes comme le nom ou le courriel, il vaut mieux exposer une vue restreinte plutôt que la table entière. Le règlement impose par ailleurs de tenir un registre des traitements de données personnelles, que j'ai rédigé et dont voici le contenu.

Traitement	Finalité	Données concernées	Base légale	Conservation	Destinataires
Gestion des commandes	exécuter et suivre les commandes	identité, courriel, adresse, montants	exécution du contrat	5 ans	logistique, comptabilité
Gestion des retours	traiter les remboursements	client, produit, montant	exécution du contrat	5 ans	service après-vente, comptabilité
Analyse des visites	mesurer l'audience du site	client, appareil, pages vues	intérêt légitime	13 mois	équipe analytique
Reporting analytique	piloter l'activité	agrégats et référence client	intérêt légitime	durée de vie du compte	équipe reporting
Historisation du client	connaître l'état du client au moment des faits	identité, ville, segment	intérêt légitime	5 ans	équipe analytique

La procédure de tri et de purge

J'ai enfin rédigé une procédure d'anonymisation des clients inactifs. Le critère retenu est l'absence de commande depuis trente-six mois. Les champs identifiants de ces clients sont alors remplacés par des valeurs neutres, tout en conservant leur identifiant technique afin de ne pas casser les analyses agrégées. Cette anonymisation est irréversible, ce qui la distingue d'une simple pseudonymisation, et elle est prévue pour s'exécuter automatiquement chaque mois. Un point important est que cette purge doit viser la source des données, car l'entrepôt est reconstruit à partir de cette source à chaque exécution. Un essai à blanc a permis d'identifier dix-sept clients concernés sans rien modifier, comme le montre l'annexe F.

9. Documenter et préparer la montée en charge

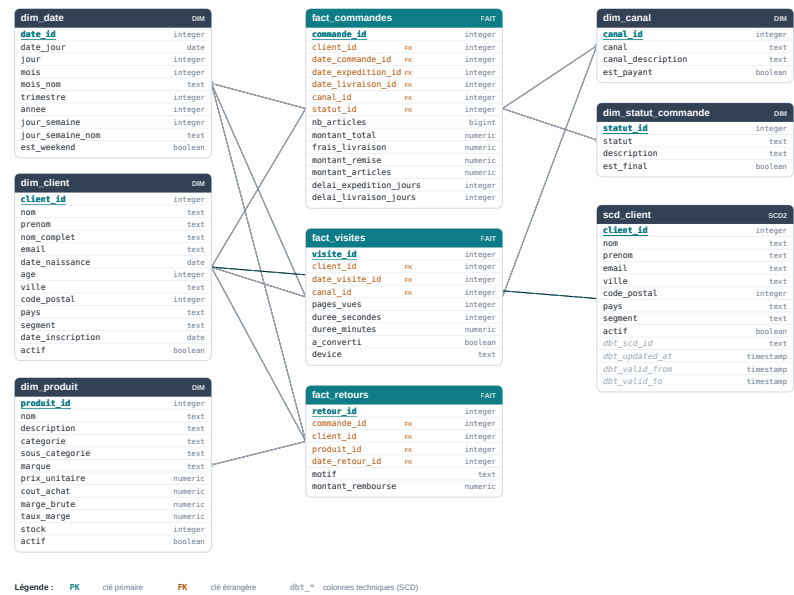
La documentation prépare l'avenir de l'entrepôt. J'ai produit un ensemble de procédures organisées par cas d'usage, afin qu'un tiers puisse comprendre et reproduire chaque opération courante. Elles couvrent la création d'un accès dans le fichier `procedure_ajout_acces.md`, l'augmentation de l'espace de stockage et de la capacité de calcul dans `procedure_stockage.md`, et l'ajout d'un nouveau datamart dans `procedure_datamart.md`. Le volet conformité est détaillé dans `rgpd_registre_et_purge.md`, et l'organisation de la maintenance dans `organisation_maintenance.md`.

Le terme datamart mérite une explication. Il s'agit d'un sous-ensemble thématique de l'entrepôt, dédié à une équipe précise comme le marketing, qui expose des tables déjà agrégées et prêtes à l'emploi. Il se construit au-dessus des faits et des dimensions existants et réutilise le schéma en étoile sans le dupliquer. Pour le stockage, la démarche documentée consiste à surveiller la taille de la base puis à agir selon le cas, par un nettoyage, une extension du disque ou l'ajout d'un espace de stockage dédié.

J'ai par ailleurs mis à jour la documentation technique générée par dbt, puis régénéré le graphe de lignage. Ce graphe montre les dépendances entre les modèles et confirme que la nouvelle source des retours est bien reliée à l'entrepôt, du fichier brut jusqu'à la table de faits. Il est présenté en annexe B.

10. Les modèles logique et physique

Le schéma ci-dessous présente l'entrepôt à jour. Il tient lieu à la fois de modèle logique, car il montre les entités et les relations, et de modèle physique, car il précise pour chaque table ses colonnes et leur type. Les tables de faits apparaissent au centre avec un en-tête coloré, les dimensions les entourent, les clés primaires sont soulignées et les clés étrangères sont signalées. Le lien en pointillés entre `dim_client` et `scd_client` représente l'historisation de type 2.



On y lit clairement la logique en étoile. Les faits `fact_commandes`, `fact_visites` et le nouveau `fact_retours` portent des mesures et des clés étrangères, tandis que les dimensions `dim_client`, `dim_produit` et `dim_date` sont partagées entre plusieurs faits, ce qui en fait des dimensions conformes. Cette réutilisation est ce qui garantit la cohérence des analyses à travers tout l'entrepôt.

11. Passage en production, ce qui changerait

Le dispositif décrit jusqu'ici fonctionne en local, sur une seule machine. En production, la logique resterait identique, seuls les outils deviendraient plus robustes, et il est utile de le décrire.

Tout s'exécuterait sur un serveur distant dans le cloud. La base de données serait un service géré qui assure lui-même la disponibilité et les sauvegardes. Le code serait poussé sur un dépôt en ligne puis empaqueté dans une image exécutable partout. La planification ne reposerait plus sur le cron mais sur un orchestrateur comme Airflow ou dbt Cloud, qui déclencherait les constructions, réessaierait en cas d'échec et émettrait lui-même les notifications.

Le point le plus sensible concernerait les secrets. La règle ne changerait pas, aucun secret dans le dépôt de code. Les mots de passe ne seraient plus posés dans des fichiers sur le disque mais rangés dans un gestionnaire de secrets dédié, puis injectés à l'exécution sous forme de variables d'environnement, référencés par un nom et jamais écrits en clair. Ce projet constitue ainsi la version atelier d'une chaîne industrielle, dont le passage en ligne ne modifie pas les principes mais les outille de façon plus automatisée et plus sûre.

12. Conclusion et retour d'expérience

L'ensemble des objectifs a été atteint et validé en conditions réelles. L'entrepôt a été compris, enrichi de la source des retours, doté d'une gestion complète des variations de dimensions, puis outillé pour son exploitation quotidienne, sa sécurité, sa conformité au RGPD et sa documentation.

Sur le plan des outils, dbt structure les transformations en couches, les fiabilise par ses tests et gère nativement l'historisation de type 2, tandis que PostgreSQL apporte une journalisation fine et des sauvegardes robustes. Les difficultés rencontrées, comme l'apostrophe qui cassait un test, ont été les plus formatrices, car elles m'ont amené à comprendre le fonctionnement réel des outils plutôt qu'à les contourner. L'entrepôt de VenteRapide est aujourd'hui surveillé, sauvegardé, sécurisé, documenté et prêt à évoluer.

Annexes

Les annexes regroupent les captures d'écran qui prouvent les résultats décrits dans le rapport. Elles ne sont pas comptées dans la pagination du corps du texte.

Annexe A. Mise en route de l'environnement

```
guiren-anice@guiren-anice-Cyborg-15-A12VF ~/Documents/Programation/datawarehouse_e6 $ psql --version
psql (PostgreSQL) 16.15 (Ubuntu 16.15-0ubuntu0.24.04.1)
guiren-anice@guiren-anice-Cyborg-15-A12VF ~/Documents/Programation/datawarehouse_e6 $ ./venv/bin/dbt --version
Core:
  - installed: 1.12.3
  - latest:    1.12.3 - Up to date!

Plugins:
  - postgres: 1.11.0 - Up to date!
```

Versions installées, PostgreSQL 16.15 et l'adaptateur dbt postgres 1.11.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF ~/Documents/Programation/datawarehouse_e6 $ ./venv/bin/dbt build
15:09:43 Running with dbt=1.12.3
15:09:44 Registered adapter: postgres=1.11.0
15:09:44 Found 12 models, 1 snapshot, 5 seeds, 66 data tests, 477 macros
15:09:44 Concurrency: 4 threads (target='dev')
15:09:44
15:09:44 1 of 84 START sql table model public.dim_date ..... [RUN]
15:09:44 2 of 84 START seed file public.raw_clients ..... [RUN]
15:09:44 3 of 84 START seed file public.raw_commandes ..... [RUN]
15:09:44 4 of 84 START seed file public.raw_lignes_commande ..... [RUN]
15:09:44 2 of 84 OK loaded seed file public.raw_clients ..... [INSERT 30 in 0.32s]
15:09:44 4 of 84 OK loaded seed file public.raw_lignes_commande ..... [INSERT 97 in 0.32s]
15:09:44 3 of 84 OK loaded seed file public.raw_commandes ..... [INSERT 50 in 0.32s]
15:09:44 5 of 84 START seed file public.raw_produits ..... [RUN]
15:09:44 6 of 84 START seed file public.raw_visites ..... [RUN]
15:09:44 1 of 84 OK created sql table model public.dim_date ..... [SELECT 731 in 0.34s]
15:09:44 7 of 84 START sql view model public.stg_clients ..... [RUN]
15:09:44 8 of 84 START sql view model public.stg_lignes_commande ..... [RUN]
15:09:45 6 of 84 OK loaded seed file public.raw_visites ..... [INSERT 60 in 0.21s]
15:09:45 5 of 84 OK loaded seed file public.raw_produits ..... [INSERT 30 in 0.21s]
15:09:45 9 of 84 START sql view model public.stg_commandes ..... [RUN]
15:09:45 7 of 84 OK created sql view model public.stg_clients ..... [CREATE VIEW in 0.23s]
15:09:45 10 of 84 START test not_null_dim_date_date_id ..... [RUN]
15:09:45 11 of 84 START test not_null_dim_date_date_jour ..... [RUN]
15:09:45 8 of 84 OK created sql view model public.stg_lignes_commande ..... [CREATE VIEW in 0.23s]
15:09:45 12 of 84 START test unique_dim_date_date_id ..... [RUN]
15:09:45 10 of 84 PASS not_null_dim_date_date_id ..... [PASS in 0.12s]
15:09:45 11 of 84 PASS not_null_dim_date_date_jour ..... [PASS in 0.12s]
15:09:45 12 of 84 PASS unique_dim_date_date_id ..... [PASS in 0.05s]
15:09:45 13 of 84 START test unique_dim_date_date_jour ..... [RUN]
15:09:45 14 of 84 START sql view model public.stg_visites ..... [RUN]
15:09:45 15 of 84 START sql view model public.stg_produits ..... [RUN]
15:09:45 9 of 84 OK created sql view model public.stg_commandes ..... [CREATE VIEW in 0.16s]
15:09:45 16 of 84 START test accepted_values_stg_clients_segment_Premium_Standard .... [RUN]
15:09:45 13 of 84 PASS unique_dim_date_date_jour ..... [PASS in 0.06s]
15:09:45 17 of 84 START test not_null_stg_clients_client_id ..... [RUN]
15:09:45 16 of 84 PASS accepted_values_stg_clients_segment_Premium_Standard ..... [PASS in 0.05s]
15:09:45 18 of 84 START test not_null_stg_clients_email ..... [RUN]
15:09:45 14 of 84 OK created sql view model public.stg_visites ..... [CREATE VIEW in 0.10s]
15:09:45 15 of 84 OK created sql view model public.stg_produits ..... [CREATE VIEW in 0.10s]
15:09:45 19 of 84 START test not_null_stg_clients_segment ..... [RUN]
15:09:45 20 of 84 START test unique_stg_clients_client_id ..... [RUN]
----- [ ... ] -----
15:09:46 81 of 84 PASS relationships_fact_visites_client_id_client_id_ref_dim_client_ . [PASS in 0.06s]
15:09:46 82 of 84 PASS relationships_fact_visites_date_visite_id_date_id_ref_dim_date_ [PASS in 0.05s]
15:09:46 84 of 84 PASS unique_fact_visites_visite_id ..... [PASS in 0.04s]
15:09:46 83 of 84 PASS unique_fact_commandes_commande_id ..... [PASS in 0.05s]
15:09:46
15:09:46 Finished running 5 seeds, 1 snapshot, 7 table models, 66 data tests, 5 view models in 0 hours 0 minutes and 2.33 seconds (2.33s).
15:09:46 Completed successfully
15:09:46
15:09:46 Done. PASS=84 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=84
guiren-anice@guiren-anice-Cyborg-15-A12VF ~/Documents/Programation/datawarehouse_e6 $
```

Construction complète réussie, les tests d'origine sont au vert.

Annexe B. Intégration de la source des retours

```
guiren-anice@guiren-anice-Cyborg-15-A12VF B ~/Documents/Programation/datawarehouse_e6 B PGPASSWORD=dbt_password psql -h localhost -p 5433 -U dbt _user -d datawarehouse_e6 -c "
select p.nom as produit, r.motif, count(*) as nb, sum(r.montant_rembourse) as rembourse
from fact_retours r join dim_produit p on r.produit_id = p.produit_id
group by p.nom, r.motif order by nb desc limit 8;"

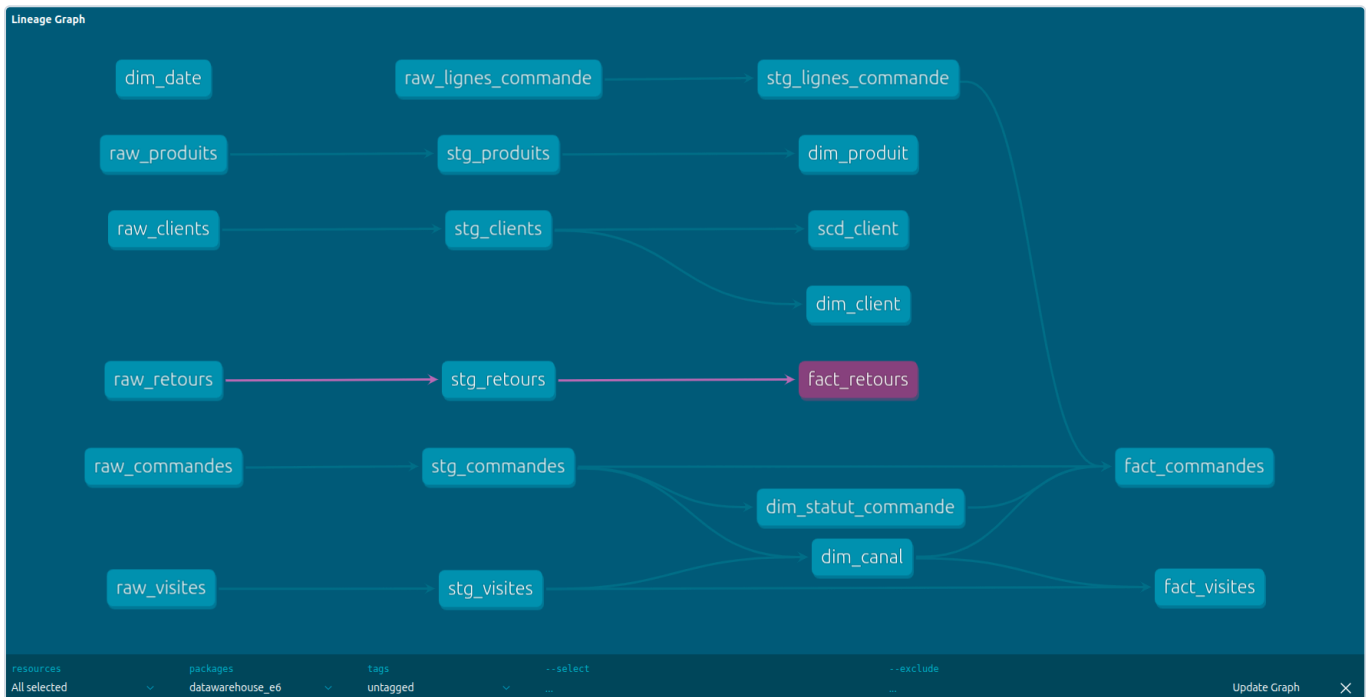
```

produit	motif	nb	rembourse
Roman Bestseller	Erreur	3	44.97
Sneakers Urban	Défaut	2	209.98
Shampooing Bio	Défaut	2	25.98
Crème Hydratante	Changement d'avis	2	65.98
Cahier A5	Erreur	1	6.99
Veste Running	Défaut	1	89.99
T-shirt Basic	Défaut	1	19.99
Casque BT Pro	Défaut	1	149.99

(8 rows)

```
guiren-anice@guiren-anice-Cyborg-15-A12VF B ~/Documents/Programation/datawarehouse_e6 B
```

Analyse rendue possible, les produits les plus retournés par motif et par montant.



Graphe de lignage, la source des retours est bien reliée jusqu'à la table de faits.

Annexe C. Variations de dimensions

```
guiren-anice@guiren-anice-Cyborg-15-A12VF B ~/Documents/Programation/datawarehouse_e6 B PGPASSWORD=dbt_password psql -h localhost -p 5433 -U dbt_user -d datawarehouse_e6 -c "select client_id, ville, code_postal, dbt_valid_from, dbt_valid_to from scd_client where client_id = 1 order by dbt_valid_from;"

```

client_id	ville	code_postal	dbt_valid_from	dbt_valid_to
1	Bordeaux	33000	2026-09-07 16:39:14.542855	2026-09-09 10:30:29.991543
1	Paris	75001	2026-09-09 10:30:29.991543	

(2 rows)

Le client 1 possède deux versions, Bordeaux archivée et Paris courante.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF B ~/Documents/Programation/datawarehouse_e6 B PGPASSWORD=dbt_password psql -h localhost -p 5433 -U dbt_user -d datawarehouse_e6 -c "
with scd as (select *, case when dbt_valid_from = min(dbt_valid_from) over (partition by client_id then '1900-01-01'::timestamp else dbt_valid_from end as vf from scd_client)
select f.commande_id, d.date_jour, s.ville as ville_au_moment, cur.ville as ville_actuelle
from fact_commandes f join dim_date d on f.date_commande_id=d.date_id
join scd s on s.client_id=f.client_id and d.date_jour>=s.vf::date and (s.dbt_valid_to is null or d.date_jour<s.dbt_valid_to::date)
join scd_client cur on cur.client_id=f.client_id and cur.dbt_valid_to is null
where f.client_id=1 order by d.date_jour;"

```

commande_id	date_jour	ville_au_moment	ville_actuelle
4	2023-01-15	Bordeaux	Paris
26	2023-06-15	Bordeaux	Paris
50	2023-12-15	Bordeaux	Paris

(3 rows)

Les commandes de 2023 ressortent avec Bordeaux, la ville d'alors, et non Paris.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF B ~/Documents/Programation/datawarehouse_e6 B PGPASSWORD=dbt_password psql -h localhost -p 5433 -U dbt_user -d datawarehouse_e6 -c "select produit_id, nom, prix_unitaire, marge_brute, taux_marge, (select count(*) from dim_produit where produit_id=1) as nb_lignes from dim_produit where produit_id = 1;"

```

produit_id	nom	prix_unitaire	marge_brute	taux_marge	nb_lignes
1	Smartphone X12	549.99	229.99	41.82	1

(1 row)

Le prix est écrasé et il ne reste qu'une seule ligne, sans historique.

Annexe D. Sauvegarde et restauration

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6 - ssh - ./scripts/backup.sh full && ./scripts/backup.sh partial && ls -lh backups/full backups/partial

OK backup full -> /home/guiren-anice/Documents/Programation/datawarehouse_e6/backups/full/dw_full_2026-09-10_1015.dump (36K)
OK backup partial -> /home/guiren-anice/Documents/Programation/datawarehouse_e6/backups/partial/dw_facts_2026-09-10_1015.dump (8,0K)
backups/full:
total 72K
-rw-rw-r-- 1 guiren-anice guiren-anice 36K sept. 9 13:52 dw_full_2026-09-09_1352.dump
-rw-rw-r-- 1 guiren-anice guiren-anice 36K sept. 10 10:15 dw_full_2026-09-10_1015.dump

backups/partial:
total 16K
-rw-rw-r-- 1 guiren-anice guiren-anice 5,5K sept. 9 13:52 dw_facts_2026-09-09_1352.dump
-rw-rw-r-- 1 guiren-anice guiren-anice 5,5K sept. 10 10:15 dw_facts_2026-09-10_1015.dump
```

Les sauvegardes complète et partielle produisent bien leurs fichiers.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6 - crontab -l
# --- Sauvegardes entrepôt VenteRapide (tache 3.3 E6) ---
PATH=/usr/bin:/bin
0 2 * * * cd /home/guiren-anice/Documents/Programation/datawarehouse_e6 && /home/guiren-anice/Documents/Programation/datawarehouse_e6/scripts/backup.sh full >> /home/guiren-anice/Documents/Programation/datawarehouse_e6/backups/backup.log 2>&1
0 */6 * * * cd /home/guiren-anice/Documents/Programation/datawarehouse_e6 && /home/guiren-anice/Documents/Programation/datawarehouse_e6/scripts/backup.sh partial >> /home/guiren-anice/Documents/Programation/datawarehouse_e6/backups/backup.log 2>&1
```

Les deux sauvegardes sont planifiées, la nuit et toutes les six heures.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6 - ./scripts/restore_test.sh
Dernier backup complet : /home/guiren-anice/Documents/Programation/datawarehouse_e6/backups/full/dw_full_2026-09-10_1015.dump
NOTICE: database "datawarehouse_e6_restore_test" does not exist, skipping
Vérification du contenu restauré :
  table | count
-----|-----
fact_commandes | 50
fact_retours | 25
dim_client | 30
scd_client | 31
(4 rows)

Base de test supprimée. Restauration validée.
```

Restauration testée dans une base jetable, les lignes récupérées sont comptées.

Annexe E. Journalisation et alertes

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6 - grep -E "WARNING:|ERROR:" /var/log/postgresql/postgresql-16-main.log | tail -8

2026-09-09 12:33:58.566 CEST [44057] dbt_user@datawarehouse_e6 ERROR: syntax error at or near "avis" at character 631
2026-09-09 15:39:59.202 CEST [102926] dbt_user@postgres ERROR: permission denied to create database
2026-09-10 09:32:27.108 CEST [16849] dbt_user@datawarehouse_e6_restore_test ERROR: permission denied to change default privileges
2026-09-10 11:25:52.230 CEST [389008] dbt_user@datawarehouse_e6 ERROR: division by zero
2026-09-10 11:35:20.255 CEST [40780] dbt_user@datawarehouse_e6 WARNING: TEST_E6 : ceci est une alerte de test (WARNING)
2026-09-10 11:35:20.287 CEST [40783] dbt_user@datawarehouse_e6 ERROR: relation "zzz_table_inexistante_e6" does not exist at character 15
2026-09-10 14:39:38.251 CEST [61326] dbt_user@datawarehouse_e6 ERROR: division by zero
```

Journaux de la base filtrés, les alertes et les erreurs sont catégorisées, avec la requête fautive.

[ALERTE dbt] Echec du build - entrepot VenteRapide

Boîte de réception x

anice.devtest@gmail.com

À moi ▾

Le 'dbt build' a ÉCHOUÉ (code de sortie 1) le 10/09/2026 à 11:25:55.

Projet : datawarehouse_e6

Log complet : /home/guiren-anice/Documents/Programation/datawarehouse_e6/logs/monitoring/dbt_build_2026-09-10_112550.log

----- dernières lignes du log -----

[0m09:25:54 102 of 109 PASS not_null_fact_visites_canal_id [[32mPASS [0m in 0.05s]

[0m09:25:54 106 of 109 START test relationships_fact_visites_canal_id__ref_dim_canal_ [RUN]

[0m09:25:54 105 of 109 PASS not_null_fact_visites_visite_id [[32mPASS [0m in 0.05s]

[0m09:25:54 103 of 109 PASS not_null_fact_visites_client_id [[32mPASS [0m in 0.06s]

[0m09:25:54 104 of 109 PASS not_null_fact_visites_date_visite_id [[32mPASS [0m in 0.06s]

[0m09:25:54 107 of 109 START test relationships_fact_visites_client_id__ref_dim_client_ [RUN]

[0m09:25:54 108 of 109 START test relationships_fact_visites_date_visite_id__ref_dim_date_ [RUN]

[0m09:25:54 109 of 109 START test unique_fact_visites_visite_id [RUN]

[0m09:25:54 106 of 109 PASS relationships_fact_visites_canal_id__ref_dim_canal_.. [[32mPASS [0m in 0.06s]

[0m09:25:54 107 of 109 PASS relationships_fact_visites_client_id__ref_dim_client_ [[32mPASS [0m in 0.05s]

[0m09:25:54 109 of 109 PASS unique_fact_visites_visite_id [[32mPASS [0m in 0.03s]

[0m09:25:54 108 of 109 PASS relationships_fact_visites_date_visite_id__ref_dim_date_ [[32mPASS [0m in 0.05s]

[0m09:25:54 Finished running 6 seeds, 1 snapshot, 9 table models, 87 data tests, 6 view models in 0 hours 0 minutes and 2.73 seconds (2.73s).

[0m09:25:54 [31mCompleted with 1 error, 0 partial successes, and 0 warnings: [0m

[0m09:25:54 [[31mERROR [0m]: in model _test_alerte (models/_test_alerte.sql)

[0m09:25:54 Database Error in model _test_alerte (models/_test_alerte.sql)

division by zero

compiled code at target/run/datawarehouse_e6/models/_test_alerte.sql

[0m09:25:54 compiled code at target/compiled/datawarehouse_e6/models/_test_alerte.sql

[0m09:25:54 Done. PASS=108 WARN=0 ERROR=1 SKIP=0 NO-OP=0 REUSED=0 TOTAL=109

Répondre

Transférer

Le courriel d’alerte reçu automatiquement après l’échec d’une construction, avec l’extrait du journal.

Annexe F. Accès et RGPD

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6
PGPASSWORD=reporting_password psql -h localhost -p 5433 -U reporting -d datawarehouse_e6 -c "select count(*) from fact_commandes;"

count
-----
      50
(1 row)

guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6
PGPASSWORD=reporting_password psql -h localhost -p 5433 -U reporting -d datawarehouse_e6 -c "insert into dim_produit(produit_id) values (99999);"

ERROR:  permission denied for table dim_produit
```

Le rôle reporting peut lire mais se voit refuser toute écriture.

```
guiren-anice@guiren-anice-Cyborg-15-A12VF: ~/Documents/Programation/datawarehouse_e6
PGPASSWORD=dbt_password psql -h localhost -p 5433 -U dbt_user -d datawarehouse_e6 -c "select count(*) as candidats from raw_clients where client_id not in (select distinct client_id from raw_commandes where date_commande::date >= (now() - interval '36 months')::date);"

candidats
-----
        17
(1 row)
```

Essai à blanc de l’anonymisation, dix-sept clients inactifs sont identifiés.

Annexe G. Tableau de bord de supervision

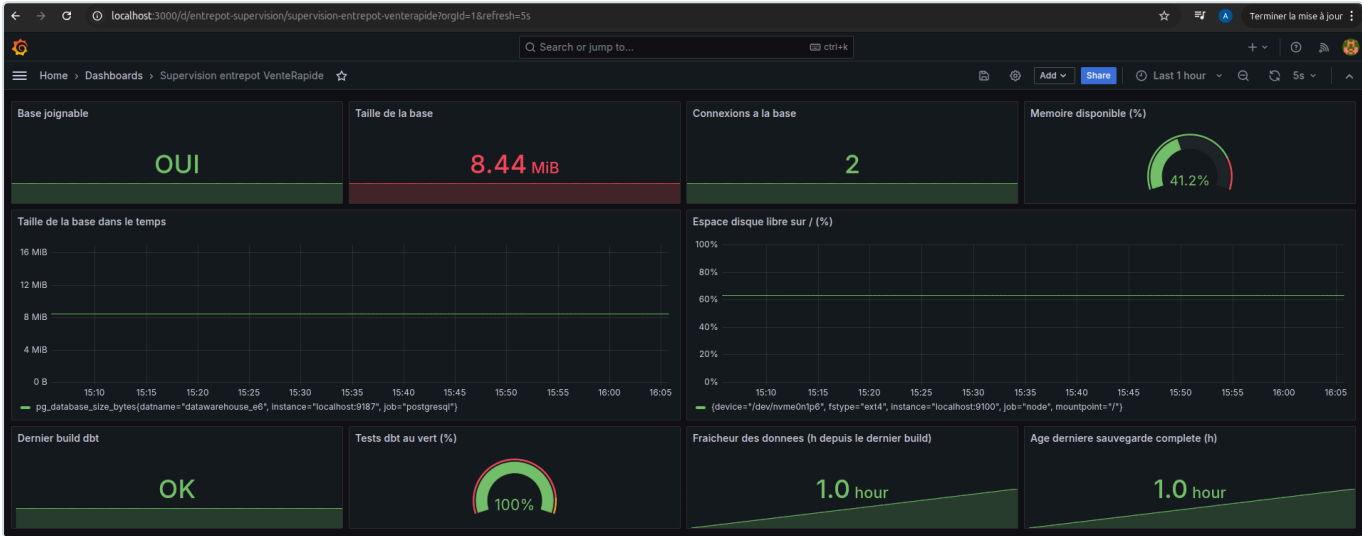


Tableau de bord Grafana, l’état de la base, de la machine, du build dbt et des sauvegardes réunis sur une seule vue, avec code couleur par seuil.